

AUTOMATIZACIÓN INDUSTRIAL

FMS-205

ÍNDICE

Recursos	5
Variables/Terminal/Canal	6
Interfaz Humano Máquina (HMI)	9
Nota importante	10
Etapas	11
Composición módulos estación:	14
1. Manual	17
2. Monolítico	19
Programa	20
Main	20
Bloque Funcional (SFC)	21
EL GRAFCET EN CUESTIÓN	23
3. Estructurado	24
FB_Estacion. Se definen las tareas y el coordinador de estas en la sección VAR:	25
Coordinador	27
Código del Coordinador.	29
4. Estructurado + Gemma	31
Proceso de funcionamiento. Procedimientos	32
Proceso en Parada o Puesta en Marcha	32
Proceso en Defecto	33
Ejemplo del código	36
DIRECTOR	40
5. Funcional	45

José Luis Pérez Martín

Ingeniería Electrónica, Robótica y Mecatrónica
Universidad de Málaga

Recursos

Manual TwinCAT 3:

https://download.beckhoff.com/download/document/automation/twincat3/TwinCAT_3_IO_EN.pdf

Interfaz de usuario (TwinCAT): <https://www.beckhoff.com/es-es/download/358059439>

TwinCAT 3 Basics: <https://www.beckhoff.com/es-es/download/682720709>

Cajas de texto

https://infosys.beckhoff.com/english.php?content=../content/1033/tc3_plc_intro/3523604491.html&id=

%d para tipo dígito

%s para string

Abreviaturas

https://infosys.beckhoff.com/english.php?content=../content/1033/tc3_plc_intro/12049871755.html&id=

Variables/Terminal/Canal

ENTRADA	T C	SALIDA
PulsadorEmergencia	02 01	
SelectorManual	02 02	
PulsadorMarcha	02 03	
PulsadorParada	02 04	
EstacionConectada	02 05	
PresenciaPale	02 06	
CodigoPaleBit0	02 07	
CodigoPaleBit1	02 08	
CodigoPaleBit2	03 01	
	04 01	DesconectaEstacion
	04 02	LamparaAlarma
	04 03	LamparaMarcha
	04 04	LamparaMaterial
	04 05	CintaActiva
	04 06	CintaInvierte
	04 07	RetenedorBaja
	05 01	AvisadorSonoro
CargadorDetras	06 01	
CargadorDelante	06 02	

CargadorArriba	06 03
CargadorAbajo	06 04
PlatoEmpujadorDetras	06 05
RechazadorDetras	06 06
RechazadorDelante	06 07
RechazadorArriba	06 08

RechazadorVacio	07 01
DescargadorDetras	07 02
DescargadorDelante	07 03
DescargadorArriba	07 04
DescargadorAbajo	07 05
AlimentadorDelante	07 06
DetectorInductivo	07 07
DetectorFotoelectrico	07 08

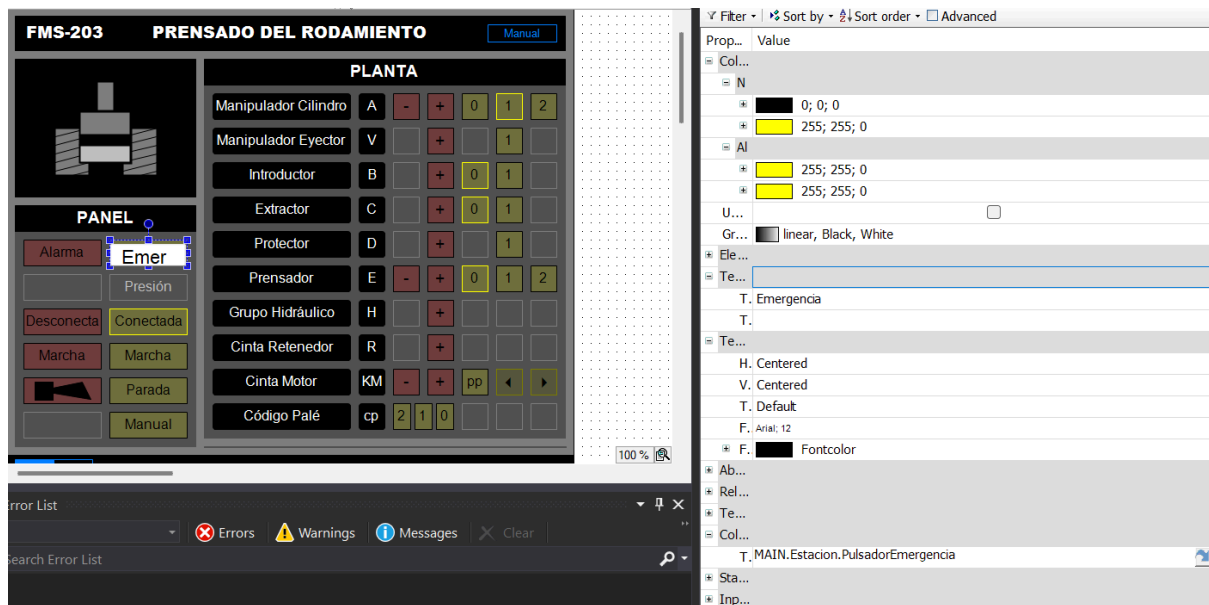
08 01	CargadorAvanza
08 02	CargadorBaja
08 03	CargadorCierra
08 04	PlatoEmpujadorAvanza
08 05	PlatoRetenedorAtras
08 06	RechazadorAvanza
08 07	RechazadorRetrocede
08 08	RechazadorBaja

	09 01	RechazadorSucciona
	09 02	DescargadorAvanza
	09 03	DescargadorBaja
	09 04	DescargadorCierra
	09 05	AlimentadorAvanza
	09 06	MedidorBaja
	09 07	VolteadorBaja
	09 08	VolteadorGira
Canal A	10 01	
Canal B	10 02	



Interfaz Humano Máquina (HMI)

- VISUs\Add\Visualization
- Crear botones, luces, etc (Toolbox: texts y colors).
 - Importante configurar “Color variables\Toggle color” → vincula variable (MAIN.Estacion.variable). [Las variables deben crearse antes de empezar con la vinculación. Si se puede hacer la interfaz]. Ejemplo:

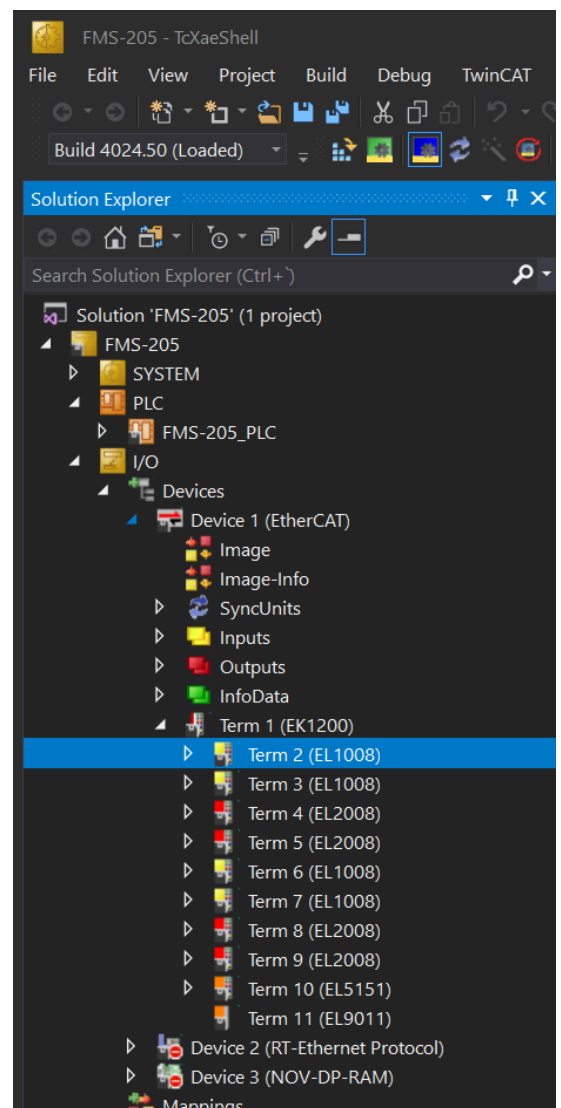


Nota importante

En la carpeta

Automatización\Ejemplos\TC3_FMS-200_Plantilla se encuentra una carpeta con la plantilla de nuestra máquina. Tiene ya la configuración de entrada y salida con los nombres (figura de la derecha). **Abrir archivo TSPROJ.**

Cada vez que se hace un cambio en alguno de estos terminales, hay que reactivar la configuración (ícono de los cubos azules y flecha amarilla)



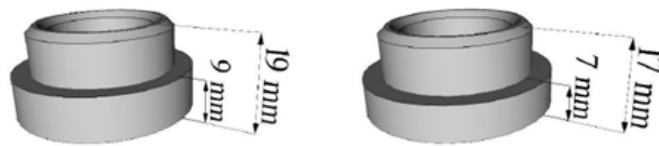
Etapas

- Las tapas pueden ser de aluminio, de nylon blanco o de nylon negro. Además, dos alturas distintas (6 combinaciones en total).
 - Necesario estipular unas condiciones iniciales (por el operario).

Estas son las ETAPAS, divididas por módulos (CUIDADO cilindros de doble efecto, hay que tener en cuenta que no vuelven solos a su punto inicial)

- **Reposo** {S0}
- **Alimentador de tapas**
 - Operario apila las tapas en el cargador (no la contamos)
 - Cilindro posiciona tapa para que el siguiente módulo la pueda coger {S1}
- **Módulo de carga** (cilindro roto-lineal y pinza neumática)
 - Desciende pinza {S2}
 - Coge tapa del módulo anterior (cierra pinzaà mantener activo) {S3}
 - Ascende pinza {S4}
 - Rota sobre sí mismo {S5}
 - Desciende pinza {S6}
 - Abre pinza (deposita la tapa en plato divisor) {...}
- Sube pinza
- Rota de vuelta
- **Plato divisor** (8 pivotes donde encajan las tapas. Gira 45° por la acción de tres cilindros [uno de tope fijo, otro móvil y un empujador que produce el giro]. 6 de los 8 tienen un módulo asociado).

- Empuja
- Vuelve
- **Medidor de tapa** (transductor digital proporciona una salida por pulsos, encoder lineal.)
 - Se extiende hasta tocar pieza o fin de carrera (indica si hay tapa, si está bien colocada [del revés] o de si no hay, en función del valor de distancia o altura obtenido de la medición).



Hay que tener cuidado porque hay dos tipos de tapas. El encoder cuenta los pulsos: el nº de pulsos es la altura (contaje rápido que hace el PLC)

- Baja
- Sube
- **Detector fotoeléctrico** (Detecta color: claro (TRUE) u oscuro (FALSE o nada))
- **Sensor inductivo** (si es metálico (TRUE) o no (FALSE))
- **Evacuación de tapas incorrectas** (2 cilindros y 3 ventosas) **INCORRECTA**
 - Cilindro horizontal avanza pinza sobre el pivote
 - Cilindro vertical desciende las ventosas
 - Eyector genera vacío (ventosas agarran pieza a mantener activo)
 - Cilindro vertical asciende
 - Cilindro horizontal retrocede pinza sobre evacuación
 - Ventosas sueltan (not eyector)
 - Horizontal

- Vertical
- **Módulo de inserción** (Igual a módulo de carga) **CORRECTA**
 - Desciende pinza
 - Coge tapa del módulo anterior (cierra pinza a mantener activo)
 - Ascende pinza
 - Rota sobre sí mismo
 - Desciende pinza
 - Abre pinza (deposita la tapa en a su salida)
 - Ascende
 - Rota

Composición módulos estación:

→→ Módulo plato divisor

●● Actuadores:

»» Cilindro empujador compacto de **doble efecto** Ø25, C:40mm (CDQ2B25-40D), con reguladores de caudal y detector de posición inicial. Controlado por electroválvula 5/2 monoestable.

»» Topes: 2 Cilindros compactos Ø16, C:10mm (CQ2B16-10D). Controlados por electroválvula 5/2 monoestable.

●● Sensores:

»» Detector magnético tipo Reed (D-A73L).

→→ Módulo alimentación de tapas

●● Capacidad almacén: 19 tapas.

●● Actuadores:

»» Cilindro empujador de **doble efecto** Ø16, C:100mm (CD85N16-100B), con reguladores de caudal y detector de posición final. Controlado por electroválvula 5/2 monoestable.

●● Sensores:

»» Detector magnético tipo Reed (D-C73L).

»» Sensor de presencia Microrruptor OMRON (V-166-1C5).

→→ Módulo estación de carga

●● Actuadores:

»» Cilindro compacto de movimiento lineal y rotativo Ø32, C:25mm (EMRQBS32-25CB), con reguladores de caudal y detector de posición inicial y final en movimiento lineal y del 0° y 180° en el rotativo. Controlado por dos electroválvulas 5/2 monoestable.

»» Brazo de sujeción: Pinzas neumáticas de dos dedos de apertura paralela (MHKL2-16D). Controladas por electroválvula 5/2 monoestable.

●●Sensores:

»» Detectores magnéticos tipo Reed (D-A73CL).

→→ **Módulo estaciones de detección del material**

●●Sensores:

»» Detector inductivo OMRON (E2A-M18KS08-WP-B1).

»» Detector capacitivo OMRON (E2K-C25MF1).

»» Detector fotoeléctrico OMRON (E3F2-DS30B4).

Módulo medición tapa

●●Actuadores:

»» Cilindro con lectura de carrera **doble efecto** Ø20, C:50mm (CE1B20-50), con reguladores de caudal. Controlado por electroválvula 5/2 monoestable.

●●Sensores:

»» Encoder lineal integrado en el cilindro.

→→ **Módulo evacuación tapa incorrecta**

●●Actuadores:

»» Eje horizontal: Cilindro de vástagos paralelos **doble efecto** Ø15, C:100mm (CXSM15-100), con reguladores de caudal y detectores de posición inicial y final. Controlado por electroválvula 5/2 biestable.

»» Eje vertical: Cilindro de vástagos paralelos y **doble efecto** Ø10, C:50mm (CXSM10-50), con reguladores de caudal y detector de posición inicial. Controlado por electroválvula 5/2 monoestable.

»» Brazo de sujeción: 3 ventosas Ø8 (ZPT08UN-B5), con eyector de generación de vacío (ZU07S). Controlado por electroválvula 3/2 monoestable.

●●Sensores:

»» Detectores magnéticos tipo Reed (D-Z73L).

»» Vacuostato de salida PNP (PS-1100-R06L).

→→ **Módulo inserción tapa**

●●Actuadores:

»» Cilindro compacto de movimiento lineal y rotativo Ø32, C:25mm (EMRQBS32-25CB), con reguladores de caudal y detectores de posición inicial y final en movimiento lineal y del 0° y 180° en el rotativo. Controlado por dos electroválvulas 5/2 monoestable.

»» Brazo de sujeción: Pinzas neumáticas de dos dedos de apertura paralela (MHKL2-16D). Controladas por electroválvula 5/2 monoestable.

●●Sensores:

»» Detectores magnéticos tipo Reed (D-A73CL).

1. Manual

```
1 PROGRAM MAIN
2 // FMS-203 - Manual
3
4 VAR
5     // Bloques funcionales
6     Estacion: FB_Estacion;
7 END_VAR
8
```

```
1 Estacion();
```

El «Main» llama al bloque funcional “Estación” (una vez por ciclo, 10 ms. Tener esto en cuenta para no caer en un bucle infinito que “pete” la máquina).
En el bloque funcional, se definen las variables (comunes a todas las máquinas y específicas de la nuestra:

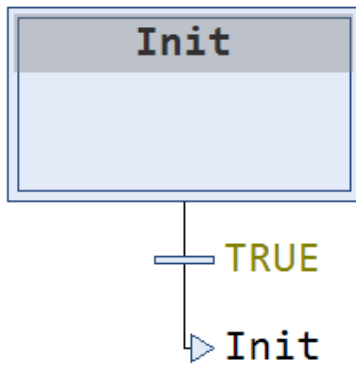
```
// Entradas comunes
PulsadorEmergencia AT %I*: BOOL;
SelectorManual AT %I*: BOOL;
PulsadorMarcha AT %I*: BOOL;
PulsadorParada AT %I*: BOOL := TRUE; // Contacto normalmente cerrado
EstacionConectada AT %I*: BOOL := TRUE;
DetectorPresion AT %I*: BOOL := TRUE;

PresenciaPale AT %I*: BOOL;
CodigoPaleBit0 AT %I*: BOOL;
CodigoPaleBit1 AT %I*: BOOL;
CodigoPaleBit2 AT %I*: BOOL;

// Salidas comunes
DesconectaEstacion AT %Q*: BOOL;
LamparaAlarma AT %Q*: BOOL;
LamparaMarcha AT %Q*: BOOL;
AvisadorSonoro AT %Q*: BOOL;

CintaActiva AT %Q*: BOOL;
CintaInvierte AT %Q*: BOOL;
RetenedorBaja AT %Q*: BOOL;
```

Y el programa del FB es:



Nosotros somos los controladores (según los parámetros de entrada, actuamos en consecuencia). Por ejemplo, si el sensor detecta que la pieza es metálica, nosotros tendremos que decidir si desecharla o continuar con el proceso.

2. Monolítico

- Objetivos

- Implementar la secuencia completa de funcionamiento de la estación (producción normal) en un único componente

- Incluir las siguientes funcionalidades

- Modos de funcionamiento: automático, manual, ciclo a ciclo, pausa y reinicio

- Gestión de la tarea (tipo de piezas y piezas solicitadas, pendientes, realizadas y rechazadas)

- Incluye la posibilidad de configurar los parámetros de tiempo de la estación

- Consideración de las condiciones iniciales

- Componentes

- PROGRAMAS

- MAIN: programa principal (ST)

- BLOQUES FUNCIONALES

- FB_Estacion: producción normal (SFC)

- FB_Clock: generador de onda cuadrada (ST)

- VISUALIZACIONES

- Visualizacion: interfaz hombre-máquina

- Procedimiento

- Modificar el programa principal (MAIN) para implementar el modo manual

- Implementar, poco a poco, la secuencia completa de funcionamiento en producción normal (FB_Estación)

- Tener en cuenta las condiciones iniciales

- Incluir los elementos de señalización (lámparas y avisador acústico)

- Incluir el control de la tarea

- Incluir la gestión del modo de funcionamiento ciclo-a-ciclo

- Considerar las situaciones ocasionales (falta material,...)

- Incluir las variables implícitas correspondientes a los indicadores SFC (SFCPause y SFCReset)

- Incluir las variables correspondientes al control de la tarea

- Incluir en la visualización los elementos necesarios para

- Controlar la tarea

- Los modos de funcionamiento

- Parámetros de funcionamiento (tiempos)
- * No tener en cuenta las situaciones excepcionales (errores) de funcionamiento de la estación

Programa

Main

```
1  PROGRAM MAIN
2  // FMS-203 - Monolitico
3
4  VAR
5      // Variables Locales
6     CodigoPale: BYTE;
7
8      // Bloques funcionales
9      Estacion: FB_Estacion;
10 END_VAR

```

```
1  IF NOT Estacion.SelectorManual THEN
2      Estacion();
3      CodigoPale := Estacion.CodigoPale;
4  ELSE
5      CodigoPale.0 := Estacion.CodigoPaleBit0;
6      CodigoPale.1 := Estacion.CodigoPaleBit1;
7      CodigoPale.2 := Estacion.CodigoPaleBit2;
8  END_IF

```

Ahora se añade la variable tipo Byte “CodigoPale”.

En el apartado de instrucciones, cuando **SelectorManual** es **FALSE**, se llama al bloque funcional **Estación**. Entonces, Se define la variable código palé (Se actualiza el valor con el estado en el que se encuentra, las **Condiciones Iniciales**).

Bloque Funcional (SFC)

- Tiene sus variables input (No son entradas)

VAR_INPUT

```
// SFC-Flags
SFCPause: BOOL;
SFCReset: BOOL;

// Parametros de control de la estacion
CicloACiclo: BOOL;

// Parametros de control de la tarea
UnidadesSolicitadas: UINT := 1;
TipoUnidadSolicitada: BYTE; // 0 = rodamiento bajo; 1 = rodamiento alto

// Parametros de tiempo
TpoCintaRetenedor: TIME := T#1S;
TpoPaleEntrada: TIME := T#400MS;
TpoPaleSalida: TIME := T#1s;
TpoPaleTransferencia: TIME := T#1S;

TpoPrensaHidraulico: TIME := T#2s;
TpoPrensaProtector: TIME := T#2s;
```

Dentro para interrupciones (FLAGS), auxiliar (control) y constantes (lo que quiere que dure cada proceso → **Nos piden procesos de duración variable/ajustable**).

- En output, las variables de entrada

VAR_OUTPUT

```
// Parametros de; estado de la estacion
CondicionInicial: BOOL;
CondicionMarcha: BOOL;
CodigoPale: BYTE;

// Parametros de control de la tarea
UnidadesPendientes: UINT;
UnidadesRealizadas: UINT;

// Variables de entrada
PulsadorEmergencia AT %I*: BOOL;
SelectorManual AT %I*: BOOL;
PulsadorMarcha AT %I*: BOOL;
PulsadorParada AT %I*: BOOL := TRUE; // Contacto normalmente cerrado
EstacionConectada AT %I*: BOOL := TRUE;
PresenciaPale AT %I*: BOOL;
CodigoPaleBit0 AT %I*: BOOL;
CodigoPaleBit1 AT %I*: BOOL;

CodigoPaleBit2 AT %I*: BOOL;
DetectorPresion AT %I*: BOOL := TRUE;

ManipuladorDetras AT %I*: BOOL; // Sobre el pale
ManipuladorMedio AT %I*: BOOL := TRUE; // En la vertical
ManipuladorDelante AT %I*: BOOL; // Sobre el punto de trasvase de la presa
IntroduccionDetras AT %I*: BOOL := TRUE;
IntroduccionDelante AT %I*: BOOL;
```

Añade un nuevo bloque: CLOCK

Sirve para crear un parpadeo (por ejemplo, para las lámparas)

```
VAR_INPUT
  EN: BOOL := TRUE; (* Enabled *)
  PT: TIME := T#500ms; (* Semicycle time *)
END_VAR
VAR_OUTPUT
  ET: TIME; (* Elapsed time *)
  Q: BOOL; (* Square wave *)
END_VAR
VAR
  Timer: TON; (* Auto-pilot timer *)
END_VAR

(* OUTPUT FUNCTION *)
IF Timer.Q THEN
  Q := NOT Q;
END_IF

(* FUNCTION BLOCKS *)
Timer(IN := EN AND NOT Timer.Q, PT := PT, ET => ET);
```

- En VAR (general) El bloque Clock y las variables de salida.

```
VAR
  E_Rodamientos: (RODAMIENTO_BAJO, RODAMIENTO_ALTO);

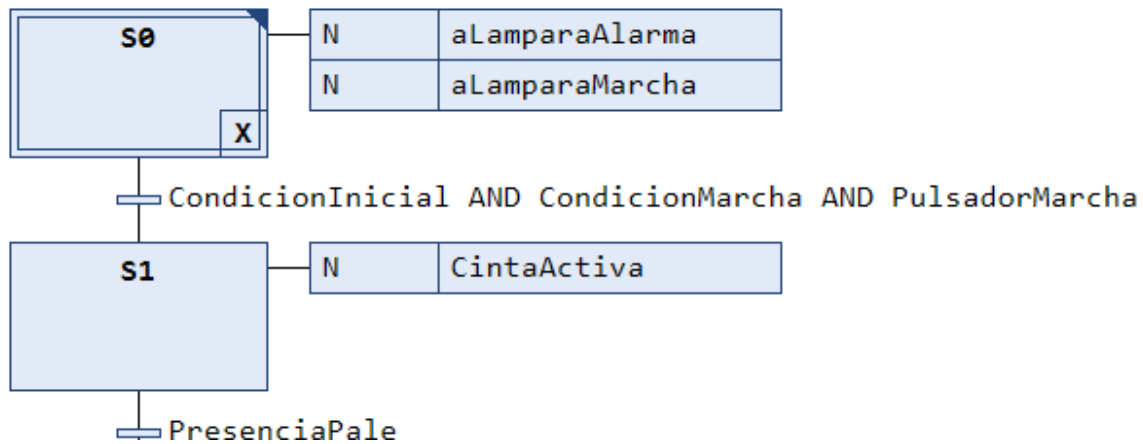
  // Bloques funcionales
  CLK: FB_Clock;

  // Variables de salida
  DesconectaEstacion AT %Q*: BOOL;
  LamparaAlarma AT %Q*: BOOL;
  LamparaMarcha AT %Q*: BOOL;
  CintaActiva AT %Q*: BOOL;
  CintaInvierte AT %Q*: BOOL;
  RetenedorBaja AT %Q*: BOOL;

  AvisadorSonoro AT %Q*: BOOL;

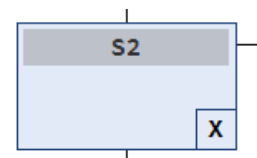
  ManipuladorAvanza AT %Q*: BOOL; // Hacia el punto de trasvase de la prensa
  ManipuladorRetrocede AT %Q*: BOOL; // Hacia el pale
  IntroductorAvanza AT %Q*: BOOL;
  ExtractorAvanza AT %Q*: BOOL;
  ProtectorBaja AT %Q*: BOOL;
  PrensadorBaja AT %Q*: BOOL;
  PrensadorSube AT %Q*: BOOL;
  ManipuladorSucciona AT %Q*: BOOL;
```

EL GRAFCET EN CUESTIÓN

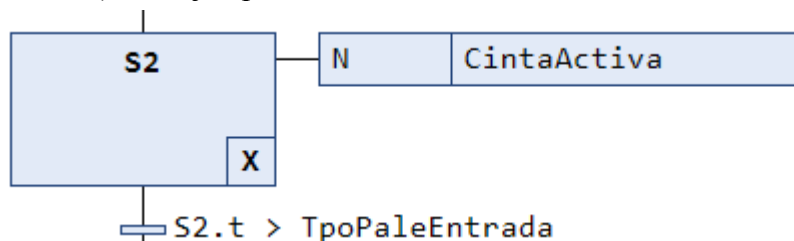


Aquí se añaden todas las posibles ETAPAS en los que se puede encontrar la máquina (S0, S1...), ACCIONES (Lo que se hace), CONDICIONES (p.e. CondicionInicial AND CondicionMarcha...)

- Las acciones “N” son continuas (durante toda la etapa)
- Hay otras que son memorizadas (definir o actualizar variables). La x de la etapa (esquina inferior derecha). Dos tipos de memorizadas
 - X es de salida
 - E es de entrada



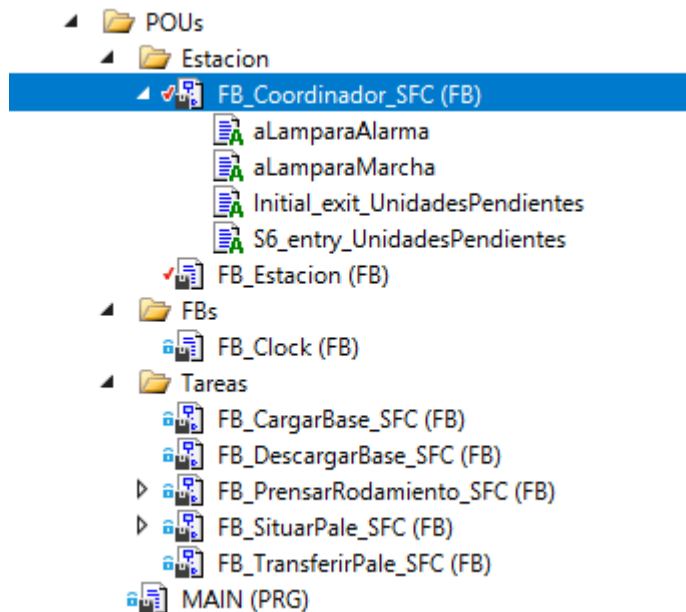
Las etapas tienen variables (.x para estado de actividad y .t para tiempo transcurrido en actividad). Por ejemplo:



Mientras que la etapa esté activa, la cinta lo está. Pasa a la siguiente etapa cuando lleve en la etapa más tiempo que la constante definida TipoPaleEntrada (400ms).

Las acciones y las transiciones más complejas se elaboran a parte (click derecho sobre el bloque funcional, add → Action/Transition).

3. Estructurado



Dentro del POU tenemos ahora el MAIN: una carpeta de la Estación y el Coordinador; otra para otros bloques funcionales (como el reloj); y una última carpeta que son las tareas.

MAIN. Muy sencillo:

```
PROGRAM MAIN
// FMS-203 - Estructurado

VAR
    Estacion: FB_Estacion;
END_VAR

Estacion();
```

Se elimina lo de “Pale-Bit”.

FB_Estacion. Se definen las tareas y el coordinador de estas en la sección *VAR*:

```
VAR
    E_Rodamientos: (RODAMIENTO_BAJO, RODAMIENTO_ALTO);

    // Bloques funcionales
    Coordinador: FB_Coordinador_SFC;
    SituarPale: FB_SituarPale_SFC;
    CargarBase: FB_CargarBase_SFC;
    PrensarRodamiento: FB_PrensarRodamiento_SFC;
    DescargarBase: FB_DescargarBase_SFC;
    TransferirPale: FB_TransferirPale_SFC;

    // Variables de salida
    DesconectaEstacion AT %Q*: BOOL;
    LamparaAlarma AT %Q*: BOOL;
```

El resto parece ser parecido (variables de entrada y salida y otras de control del proceso).

```
VAR_INPUT
    CicloaCiclo: BOOL;
    PausaEstado: BOOL;
    ReiniciaEstado: BOOL;
    UnidadesSolicitadas: UINT := 1;
    TipoUnidadSolicitada: BYTE; // 0 =

VAR_OUTPUT
    CondicionInicial: BOOL;
    CondicionMarcha: BOOL;
   CodigoPale: BYTE;
    UnidadesPendientes: UINT;
    UnidadesRealizadas: UINT;
```

En cuanto a su programa (instrucciones), se define un modo automático (“no-manual”). Supongo que Llama al Coordinador **guardando el estado** de las variables en el momento en el que se llama (del paso de manual a automático), lo que deberían ser las variables de **entrada**. Por eso se generan unas nuevas variables (**ReiniciaEstado y PausaEstado**). Se resumen algunas condiciones (como **Execute**).

También hay un control de si las tareas han terminado o si siguen en proceso (.Ready y .Done). Es decir, **parámetros de sincronización de entrada (Akc, execute) y salida (ready, done) para comunicar las tareas (rutinas) y el coordinador**.

```
// MODO AUTOMATICO
IF NOT SelectorManual THEN
  // Bloques funcionales
  Coordinador(
    UnidadesPendientes := UnidadesPendientes,
    UnidadesRealizadas := UnidadesRealizadas,
    SFCPause := NOT ReiniciaEstado AND PausaEstado,
    SFCReset := ReiniciaEstado,
    Ack := TRUE,
    Execute := CondicionInicial AND CondicionMarcha AND PulsadorMarcha,
    Resume := PulsadorMarcha,
    StepByStep := CicloaCiclo,
    PaleSituado := SituarPale.Done,
    BaseCargada := CargarBase.Done,
    RodamientoPrensado := PrensarRodamiento.Done,
    BaseDescargada := DescargarBase.Done,
    PaleTransferido := TransferirPale.Done,
    CondicionInicial := CondicionInicial,
    UnidadesSolicitadas := UnidadesSolicitadas
  );
```

Luego se hace una comunicación específica entre cada tarea y el coordinador:

```
SituarPale(
  SFCPause := NOT ReiniciaEstado AND PausaEstado,
  SFCReset := ReiniciaEstado,
  Ack := NOT Coordinador.SituaPale,
  Execute := Coordinador.SituaPale,
  TpoPaleEntrada := TpoPaleEntrada,
  PresenciaPale := PresenciaPale,
 CodigoPaleBit0 := CodigoPaleBit0,
  CodigoPaleBit1 := CodigoPaleBit1,
  CodigoPaleBit2 := CodigoPaleBit2
);

CargarBase(
  SFCPause := NOT ReiniciaEstado AND PausaEstado,
  SFCReset := ReiniciaEstado,
  Ack := NOT Coordinador.CargaBase,
  Execute := Coordinador.CargaBase,
  ManipuladorDetras := ManipuladorDetras,
  ManipuladorMedio := ManipuladorMedio,
  ManipuladorDelante := ManipuladorDelante,
  ManipuladorVacio := ManipuladorVacio
);
```

etc.

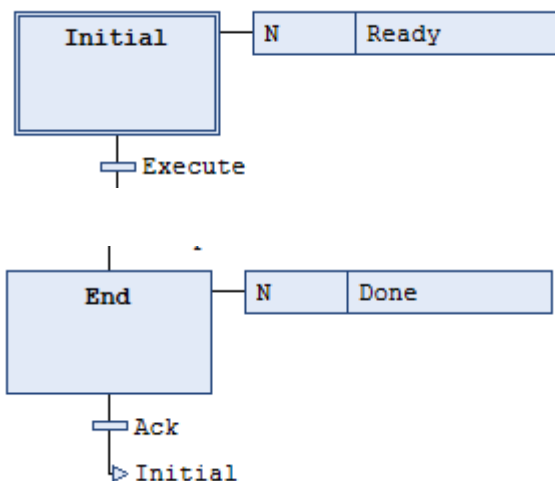
"Ack" o "Acknowledgment" (Aviso de recepción, **confirmación de evento, reconocimiento**) se refiere a un mecanismo para confirmar la recepción de una señal o dato. Es una forma de garantizar que la comunicación entre diferentes elementos o dispositivos dentro de un sistema TwinCAT ha sido exitosa.

“done”. Finalización de una tarea o el éxito de una operación. Por ejemplo, cuando se activa una tarea en TwinCAT, el parámetro "done" puede indicar si esa tarea ha sido completada o si el sistema ha llegado a su fin.

"ready" generalmente indica si un bloque funcional o una operación está **listo para ejecutarse (y por tanto, no está ejecutándose)**. Es un estado de preparación que refleja si el sistema o función está en una condición adecuada para realizar su tarea.

"execute" es utilizado principalmente para **iniciar la ejecución** de una operación o acción en un bloque funcional. Es un parámetro de entrada booleana que indica cuándo debe comenzar la operación de un bloque funcional.

En resumen, Ready (salida) y Execute (entrada) van juntos. Igualmente, Done y Ack.



Coordinador

```

VAR_IN_OUT
  UnidadesPendientes: UINT;
  UnidadesRealizadas: UINT;
END_VAR
VAR_INPUT
  // SFC-Flags
  SFCPause: BOOL;
  SFCReset: BOOL;

  // Indicadores de sincronizacion
  Ack: BOOL := TRUE;
  Execute: BOOL;

  // Indicadores de control
  Resume: BOOL;
  StepByStep: BOOL;

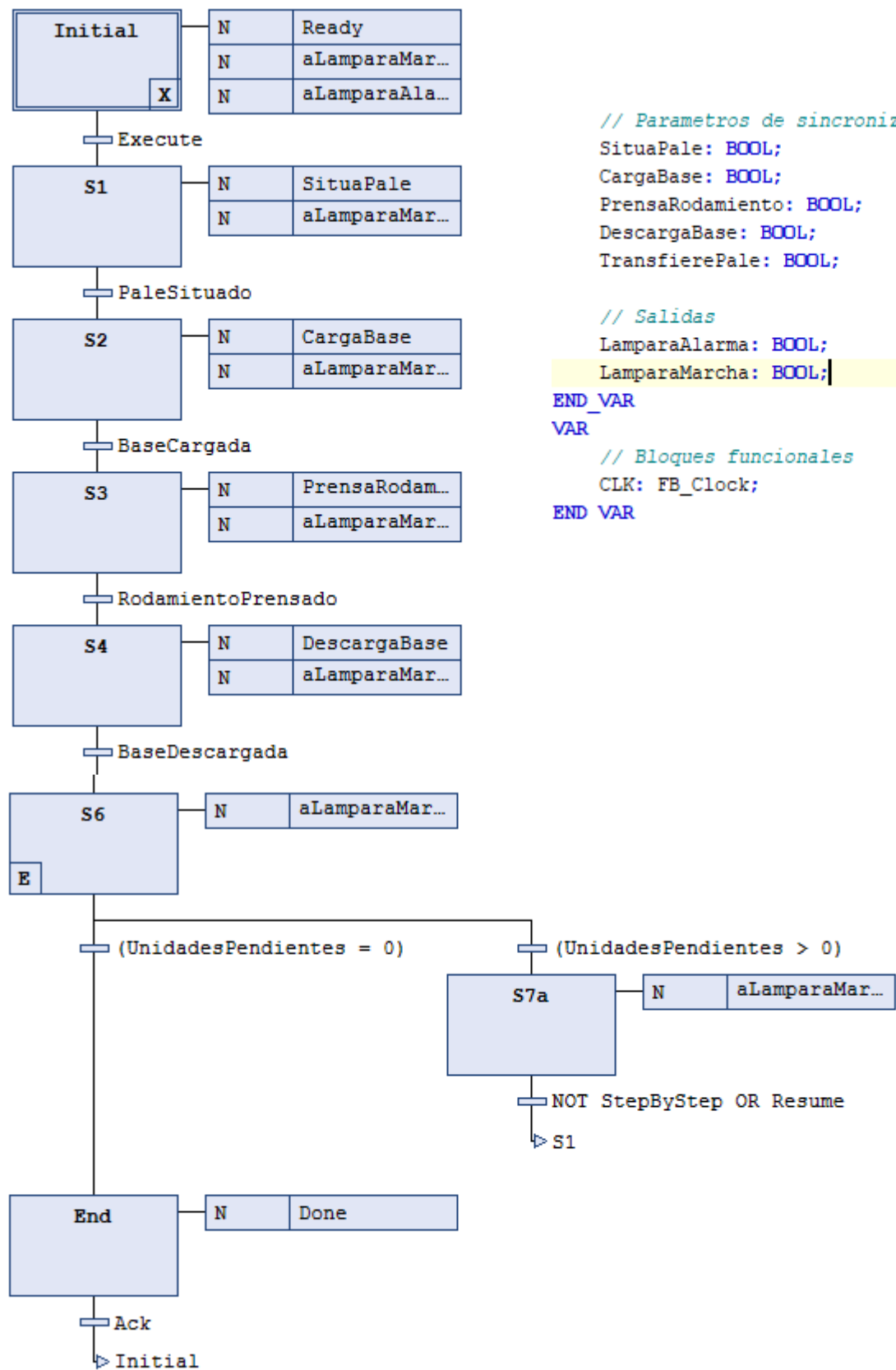
```

```

// Parametros de sincronizacion
PaleSituado: BOOL;
BaseCargada: BOOL;
RodamientoPrensado: BOOL;
BaseDescargada: BOOL;
PaleTransferido: BOOL;

// Parametros
CondicionInicial: BOOL;
UnidadesSolicitadas: UINT;
END_VAR
VAR_OUTPUT
  // Indicadores de sincronizacion
  Done: BOOL;
  Ready: BOOL;

```



// Parametros de sincronizacion

SituaPale: BOOL;

CargaBase: BOOL;

PrensaRodamiento: BOOL;

DescargaBase: BOOL;

TransfierePale: BOOL;

// Salidas

LamparaAlarma: BOOL;

LamparaMarcha: BOOL;

END_VAR

VAR

// Bloques funcionales

CLK: FB_Clock;

END VAR

Código del Coordinador.

Se ejecuta “Execute” cuando se cumple la condición inicial + condición de marcha + Pulsador de marcha (**Definido todo en el FB_Estacion**). Llama a diferentes acciones a través de los **parámetros de sincronización**.

Por ejemplo, la acción “SituaPale” hace que la variable “Coordinador.SituaPale” se ponga a TRUE mientras se encuentre en ejecución “S1”. En ese momento. El “Execute” de SituarPale se pone también a TRUE:

```
SituarPale(  
    SFCPause := NOT ReiniciaEstado AND PausaEstado,  
    SFCReset := ReiniciaEstado,  
    Ack := NOT Coordinador.SituaPale,  
    Execute := Coordinador.SituaPale,  
    TpoPaleEntrada := TpoPaleEntrada,
```

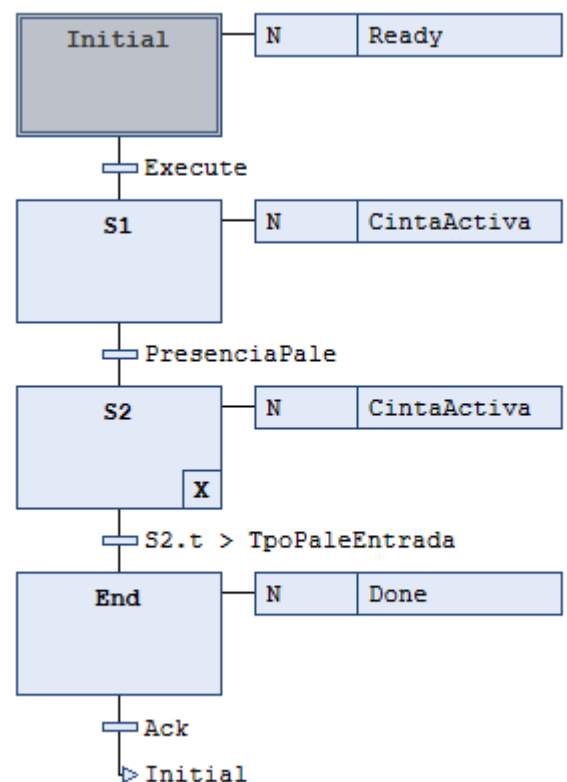
Con esto, se ejecuta todo el GRAFCET de la Tarea “SituarPale”.

Después de ejecutar todo, llega al Final y SituarPale.Done se pone a TRUE, y como consecuencia, La variable de control/Transición “PaleSituado” tmb:

```
Coordinador(  
    UnidadesPendientes := UnidadesPen  
    UnidadesRealizadas := UnidadesRea  
    SFCPause := NOT ReiniciaEstado AND  
    SFCReset := ReiniciaEstado,  
    Ack := TRUE,  
    Execute := CondicionInicial AND C  
    Resume := PulsadorMarcha,  
    StepByStep := CicloaCiclo,  
    PaleSituado := SituarPale.Done,
```

Por eso mismo, el coordinador avanza a S2, SituaPale se pone a FALSE y el Ack de Situa Palé a True. Entonces, SituarPale vuelve al estado inicial y se queda listo (Ready) para ser llamado de nuevo.

Este es el procedimiento que se sigue con cada una de las tareas.



En el Bloque funcional FB_Estacion también hay que definir las variables en función de las tareas para las acciones.

```

// Acciones
// DesconectaEstacion :=
LamparaAlarma := Coordinador.LamparaAlarma;
LamparaMarcha := Coordinador.LamparaMarcha;
// AvisadorSonoro :=

CintaActiva := NOT PausaEstado
    AND (SituarsePale.CintaActiva OR TransferirPale.CintaActiva);
// CintaMotorInvierte :=
RetenedorBaja:= TransferirPale.RetenedorBaja;
ManipuladorAvanza := CargarBase.ManipuladorAdelante
    OR DescargarBase.ManipuladorAdelante;
ManipuladorRetrocede := CargarBase.ManipuladorAtras
    OR DescargarBase.ManipuladorAtras;
IntroduccionAvanza := PrensarRodamiento.IntroduccionAdelante;
ExtractorAvanza := PrensarRodamiento.ExtractorAdelante;
ProtectorBaja := PrensarRodamiento.ProtectorBaja;
PrensadorBaja := PrensarRodamiento.PrensadorBaja;
PrensadorSube := PrensarRodamiento.PrensadorSube;
ManipuladorSucciona := CargarBase.ManipuladorSucciona
    OR DescargarBase.ManipuladorSucciona;
HidraulicoActiva := PrensarRodamiento.HidraulicoActiva;

```

4. Estructurado + Gemma

- Objetivos

- Incluir en el proyecto la gestión de los **modos de marcha y parada** (GDMMA) obtenida al aplicar a nuestro sistema la **guía GEMMA**

- La guía GEMMA (Guía de Estudio de los Modos de Marcha y Parada) es una herramienta metodológica que permite **analizar los modos de funcionamiento de un sistemas de automatización**

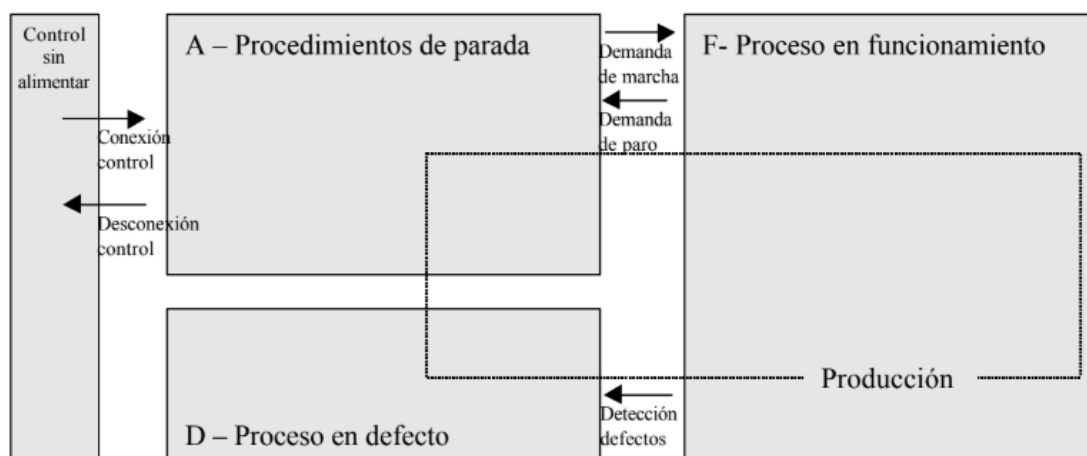
- EL GDMMA (Gráfico Descriptivo de los Modos de Marcha y Parada) resulta de aplicar la guía GEMMA a un sistema concreto. **El GDMMA resultante es una máquina de estados que describe el funcionamiento general de un sistemas de automatización**

Procedimiento

- Declarar el tipo E_GEMMA
- Incluir el módulo correspondiente al director
- Incluir los módulos correspondientes a las operaciones de restauración y preparación y finalización de la producción
- Transferir al director el control del modo manual y la gestión de la señalización del panel de operador
- Incluir en la visualización la información del modo de funcionamiento actual
- Incluir una lista de textos
- Incluir textos dinámicos para mostrar el modo de funcionamiento actual

Modos fundamentales de GEMMA

- Proceso de Funcionamiento
- Proceso en Parada o Puesta en Marcha
- Proceso en Defecto



Proceso de funcionamiento. Procedimientos

- F1 - **Producción normal**. Estado en que la máquina produce normalmente. Es el estado más importante y en él se deben realizar **las tareas** por las cuales la máquina ha sido construida.
- F2 - **Marcha de preparación**. Son las acciones necesarias para que la máquina entre en producción (precalentamiento, preparación de componentes,...).
- F3 - **Marcha de cierre**. Corresponde a la fase de vaciado y/o limpieza que en muchas máquinas debe llevarse a cabo antes de la parada o del cambio de algunas de las características del producto.
- F4 - **Marchas de verificación sin orden**. En este caso la máquina, normalmente por orden del operario, puede realizar cualquier movimiento o unos determinados movimientos preestablecidos. Es el denominado **control manual** y se utiliza para funciones de mantenimiento y verificación.
- F5 - **Marchas de verificación con orden**. En este caso la máquina realiza el ciclo completo de funcionamiento en orden pero al ritmo fijado por el operador. Se utiliza también para tareas de mantenimiento y verificación. En este estado la máquina puede estar en producción. En general, se asocia al **control semiautomático**. (Ciclo a ciclo + tarea a tarea + movimiento a movimiento)
- F6 - **Marchas de test**. Sirve para realizar operaciones de ajuste y mantenimiento preventivo, por ejemplo: comprobar si la activación de los sensores se realiza en un tiempo máximo, curvas de comportamiento de algunos actuadores, ...

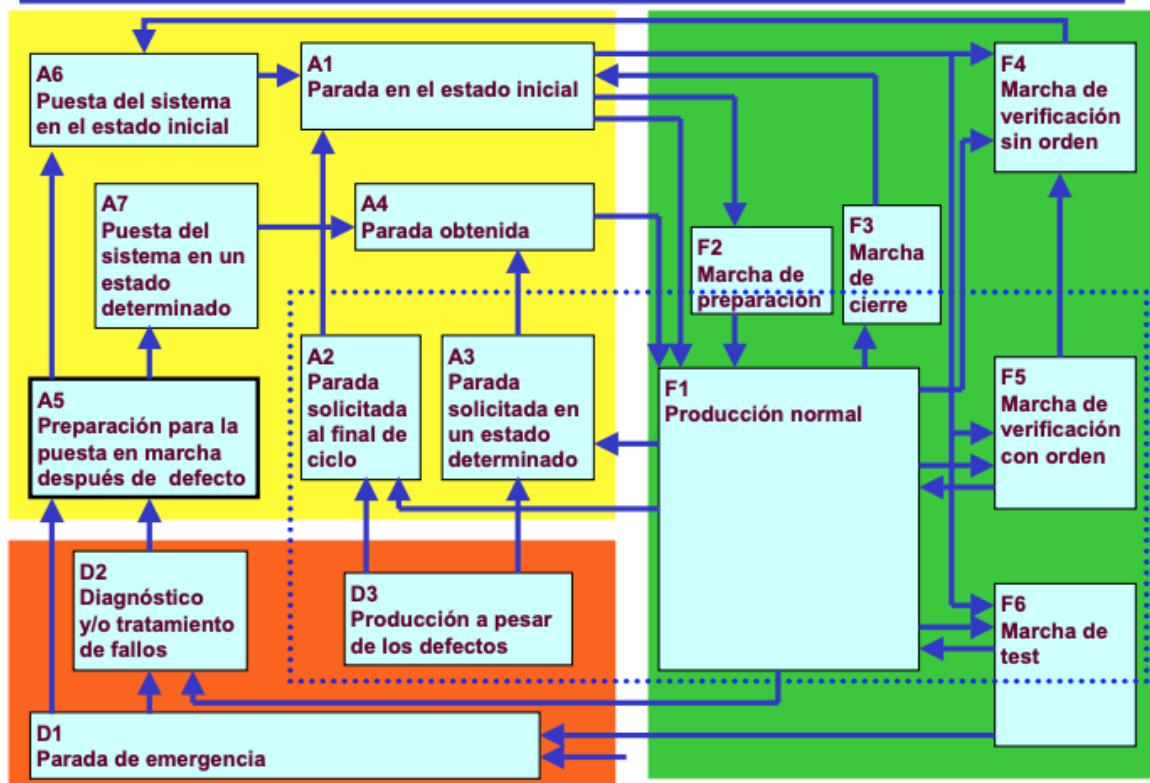
Proceso en Parada o Puesta en Marcha

- A1 - **Paradas en el estado inicial**. Se corresponde con el estado de reposo de la máquina. La máquina normalmente se representa en este estado en los planos de construcción y en los esquemas eléctricos.
- A2 - **Parada solicitada al final del ciclo**. Es un estado transitorio en que la máquina, que hasta el momento estaba produciendo normalmente, debe producir solo hasta acabar el ciclo y pasar a estar parada en el estado inicial.

- A3 - **Parada solicitada en un estado determinado.** Es un estado en que la máquina se detiene en un estado determinado que no coincide con el final de ciclo. Es un estado transitorio de **evolución hacia A4.**
- A4 - **Parada obtenida.** Es un estado de **reposo** de la máquina **distinto al estado inicial.**
- A5 - **Preparación para la puesta en marcha después de un defecto.** Es en este estado donde se procede a todas las operaciones, de: vaciado, limpieza, reposición de un determinado producto, ..., necesarias para la puesta de nuevo en funcionamiento de la máquina después de un defecto.
- A6 - **Puesta del sistema en el estado inicial.** En este estado se realiza el **retorno del sistema al estado inicial (reinicio).** El retorno puede ser manual (coincidiendo con F4) o automático.
- A7 - **Puesta del sistema en un estado determinado.** Se retorna el sistema a una posición distinta de la inicial para su puesta en marcha, puede ser también manual o automático.

Proceso en Defecto

- D1 - **Parada de emergencia.** Es el estado, que se consigue **después de una parada de emergencia,** en donde deben tenerse en cuenta tanto las paradas como los procedimientos y precauciones necesarias para evitar o limitar las consecuencias debidas a defectos.
- D2 - **Diagnóstico y/o tratamiento de fallos.** Es en este estado que la máquina puede ser examinada después de un defecto y, con ayuda o sin del operador, indicar los motivos del fallo para su rearme.
- D3 - **Producción a pesar de los defectos.** Corresponde a aquellos casos en que se deba continuar produciendo a pesar de los defectos. Se incluye en estas condiciones casos en que, por ejemplo, sea necesario finalizar un reactivo no almacenable, en que se pueda substituir transitoriamente el trabajo de la máquina por la de un operario hasta la reparación de la avería, ..



Incluimos solo los procesos que consideremos necesarios para la máquina. El resto se tachan

HAY QUE HACER un GRAFCET «maestro» y unas GRAFCET «esclavos» (tareas)
[Coordinación vertical].

Significado de los colores: Pulsadores

- **Pulsador BLANCO:**
 - Puesta en marcha/puesta en tensión.
 - En el caso de máquinas antiguas es aceptable el color VERDE.
- **Pulsador NEGRO:**
 - Parada/puesta fuera de tensión:
 - En el caso de máquinas antiguas es aceptable el color ROJO.
- **Pulsador ROJO sobre fondo AMARILLO:**
 - Parada de emergencia o iniciación de una función de emergencia
- **Pulsador AMARILLO:**
 - Supresión de condiciones anormales o restablecimiento de un ciclo automático interrumpido:
- **Pulsador AZUL.**
 - Rearme

Significado de los colores: Pilotos

- **Piloto ROJO**
 - Emergencia: condición peligrosa que requiere una acción inmediata (presión fuera de los límites de seguridad, sobrerrecorrido, rotura de acoplamiento...)
 - Demanda intervención urgente por parte del operador.
- **Piloto AMARILLO:**
 - Anomalía – condición anormal que puede llevar a una situación peligrosa (presión fuera de los límites normales, activación de un dispositivo de protección...)
 - Demanda intervención por parte del operador
- **Piloto BLANCO:**
 - Neutro – información general (presencia de tensión de red...)
- **Piloto VERDE:**
 - Máquina preparada para entrar en servicio.
 - Máquina en funcionamiento normal

Ejemplo del código

En el DUT se define el tipo enumeración “GEMMA”:

```
// Modes du Guide d'Etude des Modes de Marche et d'Arrêt
// Modes of Study Guide of the Modes of Operating and Stopping

// Procedimientos de parada [Les procédures d'arrets - Stop procedure
A1, // 0: Parada en el estado inicial
    // 0: Arrêt dans etat initial
    // 0: Stopped in the initial state
A2, // 1: Parada solicitada a final de ciclo
    // 1: Arret demandé en fin de cycle
    // 1: Requested end cycle stop
A3, // 2: Parada solicitada en un estado determinado
    // 2: Arret demande dans etat determine
    // 2: Requested particular state stop
A4, // 3: Parada obtenida
    // 3: Arret obtenu
    // 3: Obtained stop
A5, // 4: Preparacion para la recuperacion del fallo
    // 4: Preparation pour remise en route apres defaillance
    // 4: Preparation for the recovery of the failure
A6, // 5: Situando la PO en el estado inicial
    // 5: Mise PO dans état initial
    // 5: Set the PO in initial state
A7, // 6: Situando la PO en un estado determinado
    // 6: Mise PO dans état déterminé
    // 6: Set the PO in a certain state

// Procedimientos de funcionamiento
// Les procédures de fonctionnement
// Operating procedures
F1, // 7: Produccion normal
    // 7: Production normale
    // 7: Normal operation
F2, // 8: Secuencia de preparacion
    // 8: Marche de preparation
    // 8: Preparation sequence
F3, // 9: Secuencia de finalizacion
    // 9: Marche de cloture
    // 9: Ending sequence
F4, // 10: Verificacion en desorden
    // 10: Marche de verification dans le desordre
    // 10: Verification with no order
```

```

F5, // 11: Verificacion en orden
    // 11: Marche de verification dans l'ordre
    // 11: Verification whit order
F6, // 12: Secuencia de prueba
    // 12: Marche de test
    // 12: Test sequence

// Procedimientos de fallo
// es procédures en défaillances
// Failure procedures
D1, // 13: Parada de emergencia
    // 13: Arret d'urgence
    // 13: Emergency stop
D2, // 14: Diagnostico y/o tratamiento de los fallos
    // 14: Diagnostic et/ou traitement de la défaillance
    // 14: Diagnosis and/or treatment of failures
D3 // 15: Produccion a pesar de los fallos
    // 15: Production tout de même
    // 15: Production despite the failures
);
END_TYPE

```

En FB_Estacion se definen los bloques funcionales (fijese en que el reloj ahora, en vez de definirse en el Coordinador, se define en la propia estación

```

E_Rodamientos: (RODAMIENTO_BAJO, RODAMIENTO_ALTO);
ModoAnterior: E_GEMMA;

// Bloques funcionales
CLK: FB_Clock;
Director: FB_Director;
Coordinador: FB_Coordinador_SFC;
Restaurar: FB_EstacionRestaurar_SFC;
Preparar: FB_EstacionPreparar_SFC;
Finalizar: FB_EstacionFinalizar_SFC;
Situarpale: FB_CintaSituarpale_SFC;
CargarBase: FB_ManipuladorCargarBase_SFC;
PrensarRodamiento: FB_PrensaPrensarRodamiento_SFC;
DescargarBase: FB_ManipuladorDescargarBase_SFC;
TransferirPale: FB_CintaTransferirPale_SFC;

```

Se observa, además de un coordinador, un director.

```

// MODO AUTOMATICO
IF (Director.Modo <> E_GEMMA.F4) THEN
  // Bloques funcionales
  Restaurar(
    SFCPause := NOT ReiniciaEstado AND PausaEstado,
    SFCReset := ReiniciaEstado,
    Ack := TRUE,
    Execute := (Director.Modo = E_GEMMA.A6) AND PulsadorMarcha,
    ManipuladorDetras := ManipuladorDetras,
    ManipuladorMedio := ManipuladorMedio,
    PrensadorArriba := PrensadorArriba
  );

  Coordinador(
    UnidadesPendientes := UnidadesPendientes,
    UnidadesRealizadas := UnidadesRealizadas,
    SFCPause := NOT ReiniciaEstado AND PausaEstado,
    SFCReset := ReiniciaEstado,
    Ack := NOT CicloaCiclo OR PulsadorMarcha,
    Execute := (Director.Modo = E_GEMMA.F1),
    PaleSituado := SituarPale.Done,
    BaseCargada := CargarBase.Done,
    RodamientoPrensado := PrensarRodamiento.Done,
    BaseDescargada := DescargarBase.Done,
    PaleTransferido := TransferirPale.Done
  );

```

El **modo automático** es cuando el modo del director es DISTINTO del F4 de GEMMA (Es decir, es automático cuando no es manual).

El Coordinador se ejecuta cuando F1

```

Preparar(
  SFCPause := NOT ReiniciaEstado AND PausaEstado,
  SFCReset := ReiniciaEstado,
  Ack := (Director.Modo <> E_GEMMA.F2),
  Execute := (Director.Modo = E_GEMMA.F2),
);

Finalizar(
  SFCPause := NOT ReiniciaEstado AND PausaEstado,
  SFCReset := ReiniciaEstado,
  Ack := (Director.Modo <> E_GEMMA.F3),
  Execute := (Director.Modo = E_GEMMA.F3),
);

```

Se prepara con F2 y se finaliza con F3.

```

// DesconectaEstacion :=
AvisadorSonoro := ((Director.Modos = E_GEMMA.F2) AND CLK.Q)
OR (Director.Modos = E_GEMMA.F3);
LamparaAlarma := (Director.Modos = E_GEMMA.D1)
OR ((Director.Modos = E_GEMMA.A6) AND CLK.Q);
LamparaMarcha := (Director.Produccion AND NOT Coordinador.Done)
OR ((Director.Modos = E_GEMMA.A1) AND CLK.Q);

```

También se definen algunos parámetros en función del proceso GEMMA activo. Por último, lo siguiente.

```

// Al iniciar una tarea se actualiza las unidades pendientes
IF ((ModoAnterior = E_GEMMA.A1) AND (Director.Modos = E_GEMMA.F2)) THEN
    IF (UnidadesPendientes = 0) THEN
        UnidadesPendientes := UnidadesSolicitadas;
    END_IF
END_IF

IF (Director.Modos <> E_GEMMA.F4) THEN
    CodigoPale := SituarPale.PaleCodigo;
ELSE
    CodigoPale.0 := CodigoPaleBit0;
    CodigoPale.1 := CodigoPaleBit1;
    CodigoPale.2 := CodigoPaleBit2;
END_IF

// ACTUALIZACION DE IMAGENES
ModoAnterior := Director.Modos;

```

Aquí se tiene en cuenta el procedimiento activo (y el anterior) para actualizar las unidades pendientes respecto de las solicitadas o para actualizar el código palé

DIRECTOR

```
FUNCTION_BLOCK FB_Director
// FMS-203 - GEMMA

VAR_INPUT
    CondicionInicial: BOOL;
    CondicionMarcha: BOOL;
    Emergencia: BOOL;
    Finalizada: BOOL;
    FinTarea: BOOL;
    FinCiclo: BOOL;
    Manual: BOOL;
    Marcha: BOOL;
    ParaCiclo: BOOL;
    Preparada: BOOL;
    Reinicia: BOOL;
END_VAR
VAR_OUTPUT
    Modo: E_GEMMA := E_GEMMA.A6;

    Defecto: BOOL; // Procedimientos de defecto
    Funcionamiento: BOOL; // Procedimientos de funcionamiento
    Parada: BOOL; // Procedimientos de parada
    Produccion: BOOL; // Procedimientos que proporcionan valor añadido
END_VAR
```

Se definen unas variables que tomarán valor en función de la fase activa.


```

// FUNCION DE ESTADO
IF Emergencia THEN
    Modo := E_GEMMA.D1;
ELSIF Reinicia THEN
    Modo := E_GEMMA.A6;
ELSE
    CASE Modo OF
        // Procedimientos de parada
        E_GEMMA.A1: // Parada en el estado inicial
            IF NOT CondicionInicial THEN
                Modo := E_GEMMA.A6;
            ELSIF Manual THEN
                Modo := E_GEMMA.F4;
            ELSIF (CondicionInicial AND CondicionMarcha AND Marcha) THEN
                Modo := E_GEMMA.F2;
            END IF
        E_GEMMA.A2: // Parada solicitada a final de ciclo
            IF FinCiclo THEN
                Modo := E_GEMMA.A1;
            END IF
        E_GEMMA.A3: // Parada solicitada en un estado determinado
            ;
        E_GEMMA.A4: // Parada obtenida
            ;
        E_GEMMA.A5: // Preparando de reanudacion tras el fallo
            ;
        E_GEMMA.A6: // Situando la PO en el estado inicial
            IF CondicionInicial THEN
                Modo := E_GEMMA.A1;
            ELSIF Manual THEN
                Modo := E_GEMMA.F4;
            END IF
        E_GEMMA.A7: // Situando la PO en un estado determinado
            ;

        // Procedimientos de funcionamiento
        E_GEMMA.F1: // Produccion normal
            IF FinTarea THEN
                Modo := E_GEMMA.F3;
            ELSIF ParaCiclo THEN
                Modo := E_GEMMA.A2;
            END IF
        E_GEMMA.F2: // Secuencia de preparacion
            IF Preparada THEN
                Modo := E_GEMMA.F1;

```



```

        Modo := E_GEMMA.A2;
    END_IF
E_GEMMA.F2: // Secuencia de preparacion
    IF Preparada THEN
        Modo := E_GEMMA.F1;
    END_IF
E_GEMMA.F3: // Secuencia de finalizacion
    IF Finalizada THEN
        Modo := E_GEMMA.A1;
    END_IF
E_GEMMA.F4: // Verificacion sin orden
    IF NOT Manual THEN
        Modo := E_GEMMA.A6;
    END_IF
E_GEMMA.F5: // Verificacion en orden
    ;
E_GEMMA.F6: // Prueba
    ;

// Procedimientos de defecto
E_GEMMA.D1: // Parada de emergencia
    IF NOT Emergencia THEN
        Modo := E_GEMMA.A6;
    END_IF
E_GEMMA.D2: // Diagnostico/tratamiento de los fallos
    ;
E_GEMMA.D3: // Produccion a pesar de los fallos
    ;
END_CASE
END_IF

// PARAMETROS DE SALIDA
Defecto := (Modo >= E_GEMMA.D1) AND (Modo <= E_GEMMA.D3);
Funcionamiento:= (Modo >= E_GEMMA.F1) AND (Modo <= E_GEMMA.F6);
Parada := (Modo >= E_GEMMA.A1) AND (Modo <= E_GEMMA.A7);
Produccion := (Modo = E_GEMMA.A2)
    OR (Modo = E_GEMMA.A3)
    OR (Modo = E_GEMMA.F1)
    OR (Modo = E_GEMMA.F2)
    OR (Modo = E_GEMMA.F3)
    OR (Modo = E_GEMMA.F5)
    OR (Modo = E_GEMMA.F6)
    OR (Modo = E_GEMMA.D3);

```

Se define un **SWITCH (CASE)** que se activará en el caso de que no esté en modo EMERGENCIA o modo REINICIO. Aquí se definen todos los **procedimientos** (los que se vayan a emplear; algunos no son necesarios para la máquina) y los **parámetros de salida**.

Explicación, una a una, de las transiciones entre procesos GEMMA:

- **D1**, Parada de Emergencia. Se activa cuando el pulsador de emergencia es accionado.

PROCEDIMIENTOS DE PARADA

- **A6**, Reinicio, cuando se pulsa el botón de Reset.

Si no están pulsados el botón de reset o el botón de emergencia:

- **A1**. Parada en el estado inicial:
 - **A6**. Retorno al sistema si no están las condiciones iniciales
 - Si se cummplen las condiciones iniciales:
 - **F4**, se Manual está seleccionado
 - Si no, Si se cumplen las condiciones iniciales, no está manual, se cumple la condición de Marcha y se pulsa el botón de marcha, la máquina activa **F2, Marcha de preparación** (se prepara para entrar en producción).
- **A2**, si se solicita parada al final del ciclo
- **A3** (parada en estado determinado), **A4** (parada obtenida) y **A5** (preparar reanudación tras el fallo). Nada
- **A6**. PO en el estado inicial
 - Si condición inicial, **A1**
 - Si no condición inicial y manual, **F4**.
- **A7** Nada.

PROCEDIMIENTOS DE FUNCIONAMIENTO

- **F1**. Producción normal:
 - Si fin de tarea, **F3**
 - Si para ciclo, **A2**
- **F2**. Preparación; si preparada, **F1 (producción)**
- **F3**. Finalización. Si finalizada, **A1**.
- **F4**. Modo manual sin orden. Si se desactiva el Manual, **A6**.
- **F5**. Nada.
- **F6**. Nada.

PROCEDIMIENTOS DE DEFECTO

- **D1**. Parada de emergencia. Si no se pulsa emergencia, **A6**
- **D2 y D3**. Nada

// PARAMETROS DE SALIDA

```
Defecto := (Modo >= E_GEMMA.D1) AND (Modo <= E_GEMMA.D3);
Funcionamiento:= (Modo >= E_GEMMA.F1) AND (Modo <= E_GEMMA.F6);
Parada := (Modo >= E_GEMMA.A1) AND (Modo <= E_GEMMA.A7);
Produccion := (Modo = E_GEMMA.A2)
              OR (Modo = E_GEMMA.A3)
              OR (Modo = E_GEMMA.F1)
              OR (Modo = E_GEMMA.F2)
              OR (Modo = E_GEMMA.F3)
              OR (Modo = E_GEMMA.F5)
              OR (Modo = E_GEMMA.F6)
              OR (Modo = E_GEMMA.D3);
```

- **Defecto** si se encuentra en algún modo de los Ds.
- **Funcionamiento** si Fs
- **Parada** si As
- **Producción** si A2, A3, Fs o D3.

5. Funcional

Objetivos

- Crear los FB correspondientes a cada una de las funcionalidades (servicios) de cada una de las unidades funcionales (anteriormente tareas)
- Crear un FB correspondiente a la secuencia de restauración de cada una de las unidades funcionales (antes agrupados en un único diagrama)
- Crear un FB para cada controlador (ECC) de cada una de las unidades funcionales.
- Crear el FB principal de cada unidad funcional
 - Incluirá todas las variables de entrada (sensores) y salida (actuadores) de la unidad funcional
 - Las variables propias para la coordinación con otras unidades funcionales o con el coordinador de la estación
 - Las variables necesarias para la determinación de las condiciones iniciales, la gestión distribuida del modo manual (mandos directos) y la restauración de la unidad funcional
- Reorganizar el FB correspondiente a la estación
 - Se pasa de trabajar con tareas a trabajar con unidades funcionales
 - Se cede la gestión de las variables de entrada (sensores) y salida (actuadores) a las unidades funcionales
 - Se cede la gestión de la restauración y el modo manual a las unidades funcionales
- Opcionalmente, reorganizar la visualización en términos de unidades funcionales
- Incluir en la visualización los objetos necesarios para gestionar las unidades funcionales